MARS: A Memory-Augmented Rust Shell — a Terminal-Resident LLM Agent that Sees Your Workspace and Remembers

Anjishnu Kumar
anjishnu.kr@gmail.com

Abstract

LLM assistants for developers are placed *beside* the work, not inside it: a browser tab, a separate CLI, or a cloud IDE that sees only what is pasted into it, dies when its window closes, and — run against a remote GPU box — either ships source and API keys to a machine you do not trust or reasons about a host it cannot see. Two structural deficits follow: the agent is **blind** to the terminal and **amnesiac** across sessions. We present **MARS** (Memory-Augmented Rust Shell), a single-binary terminal editor, multiplexer, and built-in agent whose thesis is *memory augmentation*: an agent that lives at the terminal, sees every pane as structured context, and *remembers* — the user's own accepted commands and shell history for translation, and MARS's own documentation, action registry, and tuning-knob descriptions for self-knowledge and self-reconfiguration, retrieved by lightweight BM25. A standalone translator cannot do this: it neither sits at the terminal nor accumulates the user's context. Around this core MARS adds the *minimum* machinery to make it work: a portable, registry-validated directive protocol; corrective memory that turns a once-wrong command right and keeps it right; a model cascade that rotates open models for free-tier limits and escalates to closed frontier models only on a detected quality failure; a persistent daemon so the agent watches and briefs across detach; and a key-never-leaves-home SSH broker for keyless remote boxes. We describe the agent, audit an ML-researcher workflow with versus without it, detail the enabling architecture, and specify — but do not yet report — a two-axis, memory-ablated evaluation.

1 Introduction

An ML researcher at a terminal is almost never using one tool. They launch and babysit training runs in a shell, edit configs and model code in an editor, hold several sessions open across remote GPU boxes with a multiplexer, and — since 2023 — ask an LLM to explain a CUDA out-of-memory trace or draft an `rsync` incantation. These tools do not share state. The editor does not know what the shell printed; the multiplexer cannot reason over its own scrollback; and the assistant, living in a separate window, sees only the fragment the user copies into it. The dominant cost of the workflow is not thinking but *shuttling context*: select the stack trace, switch windows, paste, retype the surrounding situation in prose, read, switch back, find the file, jump to the line.

Two deficits make this acute exactly where researchers most need help. The agent is **blind**: it cannot see the adjacent pane's editor buffer or the failing run's scrollback, so every question begins with a copy-paste. And it is **amnesiac**: it forgets the project-specific launch command you corrected yesterday, forgets "where was I" when you return to a detached run, and knows nothing about the tool it is embedded in. Worse, in the remote setting that defines GPU work, cloud and laptop-resident tools **leak code and keys**: to reach the work they move source and credentials onto boxes the researcher does not own, and they cannot supervise a run on a host they were never on.

Thesis: memory augmentation at the terminal. MARS is built on the claim that the assistant belongs *inside the thing that holds the work*, and that its defining capability is *memory*. A terminal-resident agent can do two things a standalone translator structurally cannot, because it sits where the work is and persists alongside it: it can *see* (read every editor buffer, terminal pane, scrollback, and the layout as structured context, with no copy-paste), and it can *remember* (retrieve the user's own accumulated commands and MARS's own ground truth to answer better than a cold model). Memory augmentation is the through-line of this

paper; sight and persistence are the substrate that makes it possible.

Contributions. (1) A memory-augmented terminal agent (§2) whose two retrieval axes — over the user's commands and over MARS's own docs/registry — plus a corrective-memory loop are the NLP core, delivered through a deliberately minimal, portable, registry-validated directive protocol. (2) A persona-grounded audit (§3) of an ML-researcher workflow with versus without the agent. (3) A single-binary architecture (§4) — source-agnostic core, server-renders-ANSI session daemon, and a key-never-leaves-home SSH broker — framed as the substrate the memory-augmented agent requires. (4) A design philosophy (§5) of *tasteful minimalism* and an *honesty invariant*. (5) A fully specified, memory-ablated evaluation (§6); the harness ships in the binary, results are pending and marked TBD. MARS is open source (MIT), `cargo install mars-terminal`, ∼9,500 lines of Rust.

2 The Memory-Augmented Agent

2.1 Tasteful minimalism

MARS adds the *minimum* machinery that makes memory augmentation work, and no more. The agent is not a second subsystem bolted beside the editor; it is a fifth input device that resolves, like every other, to one canonical `Action` registry. Everything runnable is a variant of a single `Action` enum, and three interfaces resolve intent to it — a direct chord (procedural retrieval), a fuzzy command bar (recognition), and the agent (natural language) — all terminating in one dispatch point. A capability added once is therefore chord-bindable, bar-searchable, and agent-invokable at no extra cost. Minimalism here is a feature, not an absence: retrieval is lexical (BM25) rather than a vector database; the agent speaks a trailing-line text protocol rather than a bespoke tool-calling schema; memory is two append-only files. Each choice is the smallest thing that clears the bar, which keeps the system legible and its failure modes few. The rest of this section enumerates the NLP features and, for each, *how* it is achieved.

2.2 The agent as a validated input device

The agent is provider-agnostic over any OpenAI-compatible endpoint (plus a native Anthropic path), with paid-first precedence. Rather than native tool-calling it uses a **trailing-line directive vocabulary**, chosen for model portability (it works with local open models) and a human-readable confirm gate. The system prompt hands the model a live-generated catalog of every action with its description and current keybinding, and asks it to end with exactly one directive:

Directive	Meaning
`RUN: <Action>`	fire a registry action
`TYPE: <cmd>`	type a shell command into a pane
`OPEN: path:line`	jump to a file/line
`NEED: <ctx>`	request more context; re-ask once

How it stays safe: *registry validation.* A `RUN:` whose action name does not resolve against the live registry is rejected before it can fire, so a hallucinated action is structurally impossible — the same enum that defines "what MARS can do" defines what the agent may invoke. A `RUN:` on a destructive action passes the same confirmation gate a human would; agent output is otherwise inert text until the user presses Enter. `NEED:` implements a model-driven context selector: the model can pull the full scrollback or another tab on demand rather than MARS always dumping everything, and MARS supplies the source and re-asks *once* (the depth is hard-capped, so a loop cannot form). Parsing tolerates markdown noise and strips `<think>` blocks from reasoning models.

2.3 Natural-language shell translation

In a terminal pane, `Ctrl+Space` then plain English ("find the biggest checkpoints here") is translated to a shell command and shown at the cursor for confirmation — never auto-run. **How:** the request plus on-screen context (current directory, recent scrollback) is sent to the cascade (§2.6); the reply is cleaned and displayed; if the user instead typed a real MARS command it is recognized and dispatched directly, and `!` forces raw shell. The situational context and shown-before-run teaching are the edge over a context-free translator (Lin et al., 2018, 2017).

2.4 Two memory axes: the core

The heart of MARS is retrieval over the agent's *own* context — deliberately lexical (BM25, k_1=1.5, b=0.75) because the claim under test is that *sitting at the terminal* beats a generic model, not that the retriever is fancy; embeddings do not pay their complexity rent below ∼1k items (Robertson and Zaragoza, 2009; Lewis et al.,

2020). Two corpora are ranked and injected as prompt context.

Axis A — command memory (the user's context). Every accepted (`request → command`) translation is appended to `~/.mars/cmd_memory.jsonl`, and recent shell history is read from `$HISTFILE`. **How:** on a new request, BM25 ranks these against it and the top pairs are injected as few-shot exemplars. This is what a standalone translator cannot do — it never observed the user run `sbatch -gres=gpu:8 train.sh` last week, so it cannot reproduce their project-specific idiom; MARS did, and can.

Axis B — system knowledge (MARS's own context). MARS's documentation, the action registry, and the described tuning knobs are chunked and, on a `NEED: docs` pull, BM25-ranked into the `ask` prompt. **How:** because the registry and knob descriptions are generated from code, this is ground truth — the agent answers "how do I split the screen?" and "make the accent orange" from the live keymap and the actual knob, not from priors, and honestly says "MARS can't do that yet" when retrieval returns nothing.

2.5 Corrective memory: wrong → right, compounding

Command memory is not a static cache but a *corrective* loop. **How:** when the user edits or rejects a proposed command and runs a different one, the *accepted* pair is what gets stored (an outcome hook records accept/edit/reject, joined by call id). The next similar request retrieves that correction as an exemplar, so a translation that was once wrong is answered right thereafter — and the effect compounds as the store grows. This wrong-to-right lift, measured against a no-memory baseline, is the central quantity of the evaluation (§6), and it is a capability only a terminal-resident, persistent agent can accumulate.

2.6 The model cascade

Most terminal LLM traffic is cheap (naming a tab, summarizing a quiet pane: automatic, tiny, latency-insensitive, free if wrong); the quality-critical traffic (triaging a failure and proposing a possibly-destructive action) is narrow. A per-task *cascade* exploits this with two moves it never confuses. **Rotate for limits (horizontal):** every provider meters each model *family* on separate rate-limit counters, so rotating a task across N same-tier open models multiplies the effective free-tier ceiling by roughly N, triggered reactively by a `429`, strictly in-tier. **Escalate for quality (vertical):** a task starts at a complexity tier and re-runs one tier up only on a *detected* failure — and MARS detects one deterministically because its output is structured and validated: a `RUN:` naming an action absent from the registry is a free, unambiguous signal a small model erred. Three tiers fall out of the real workloads: reflex (auto-naming, watch) on fast open models; utility (shell, normal Q&A) on strong open models; deep (triage, destructive-directive contexts) reaching closed frontier models only when the work needs them. Rotation never drops below a task's quality floor.

Task	Without	With MARS
Recall a project's launch command	retype from memory or `grep` history by hand	BM25 over your own accepted commands returns it
A once-wrong command	wrong again next time	corrective memory: right thereafter
Monitor a 40-min run, then walk away	babysit, or miss the failure	watch fires in the daemon while detached
Return after a meeting	scroll panes to reconstruct state	Away Digest: what finished / failed / changed
Triage a CUDA OOM trace	copy trace to a chat tab, retype context	agent sees the pane; jump to `file:line`
Agent on a keyless GPU box	export key onto the box (leaks)	broker proxies home; key never lands
Reconfigure the tool	read docs, edit JSON by hand	"autosave more often" → validated knob edit

Table 1: The ML-researcher workflow without versus with the memory-augmented terminal agent. The recurring gain is *context that is preserved and recalled* rather than reconstructed.

3 ML-Researcher Workflow Audit

We ground the value in a concrete persona: an ML engineer training models across a laptop and several remote GPU boxes, who launches and monitors long runs, triages failures, recalls project-specific commands, and reconfigures tools. Table 1 contrasts the workflow without and with the memory-augmented terminal agent; the narrative below is honest about where the gain is real and where it is modest.

The largest, most honest win is **context preser-**

vation and recall. Command recall (rows 1–2) is where memory augmentation is decisive: the agent reproduces *your* idiom because it saw you use it, and a correction sticks. **Persistence** (rows 3–4) turns sight into across-time vigilance — a run watched while the laptop is closed, and a failures-first briefing on return — which no window-bound tool offers. **Remote safety** (row 6) is categorical: the key is structurally incapable of landing on a box the researcher does not own. We are candid that NL→shell translation itself (implicit in several rows) is contested territory; its MARS-specific edge is the memory and situational context, not the translation.

4 System Architecture

The architecture exists to make the memory-augmented agent possible; it is deliberately small.

One source-agnostic core. The application core (`App`) owns all state — buffers, panes, tabs, terminals, the agent conversation — and never reads or writes a TTY. Input arrives as source-neutral `InputEvent` values; output is a stateless render pass painting a `ratatui` backend. The identical `App` therefore runs in three configurations with no forks: standalone on a real TTY, a headless session daemon over a socket, and a test backend (§6). PTY panes (`portable-pty` + a `vt100` parser) and LLM calls run on their own threads, producing on channels that a per-frame `tick()` drains; they never depend on anyone watching.

Persistence: thin client, server renders. Session persistence is a process split, not a save/restore layer. The server runs the whole `App` headless and emits `ratatui`'s own ANSI byte stream over a Unix socket; the client owns the real TTY and is a thin pump. We chose server-renders-ANSI over a structured protocol because it reuses 100% of the rendering pipeline with no second renderer to build. Because terminal and agent threads never depended on the render loop, `tick()` keeps running with no client attached — which is exactly why a watched run keeps being watched while detached, and why the agent can accumulate memory and brief across sessions.

The sigil command bar. One gesture (`Ctrl+Space`) opens the bar, and its *first character declares the namespace*: plain text fuzzy-searches actions, `!` runs a shell command, `?` asks the agent, `@` opens the file finder. This is *not* an intent classifier — routing is deterministic sigil dispatch, so it never mispredicts and needs no model to decide where input goes (hence no classification metric to report; §6).

Key-never-leaves-home SSH broker. A secret is a fact about *you*, not a machine, yet exporting your key on every GPU box scatters it into environments, shell histories, and dotfiles you do not control. MARS relocates *where the call fires*: a home daemon (`mars keyd`) holds the key; when you `mars ssh <host>` its socket is reverse-forwarded (riding the system `ssh`, so jump hosts and hardware keys work unchanged); the remote agent serializes the request *home*, the broker injects `Authorization`, calls the provider, and streams the completion back. The key never lands on the remote — not on disk, in env, or in memory — so a compromised box has nothing to steal, and access ends the instant you detach. Because escalation happens home-side, a keyless box transparently gets frontier-quality triage.

Self-instrumentation. Running with `-llm-debug` logs every call — task, model, real token counts, latency, full prompt and reply — as one JSONL stream that is at once a cost profile (`mars llm-stats` ranks per task×model by tokens), a behavioral signal (an outcomes file records accept/edit/reject), and the benchmark corpus the evaluation draws on.

5 Design Philosophy

Tasteful minimalism (lead). Every subsystem above is the smallest mechanism that clears its bar: lexical retrieval over two flat files, a text directive protocol, one action registry serving all actors, a daemon that reuses the render pipeline. The discipline is to *not* add a vector store, a tool-calling schema, or a second renderer until a shipped feature demands it. A small, legible system has fewer failure modes and is auditable end-to-end — which is why the primary test suite can drive the real thing with no mocks.

The honesty invariant (condensed). No surface may display a keybinding it did not read from the live keymap. Every hint — status bar, dropdown, which-key panel — derives its binding text from one function at render time, so a remap updates every surface at once and no hint can lie. The invariant extends to the agent: it is fed the live

keymap and cites real bindings, and it is capability-tiered, teaching only chords the user's terminal can actually send. Trust in hints is the precondition for the editor's self-teaching pedagogy; the same discipline makes the agent's citations trustworthy.

6 Evaluation

The evaluation is specified in full; the harness ships in the binary (the memory-mode flag of §2.4 and the call log of §4). The study is not yet run, so all numbers are placeholders marked TBD.

What we do and do not measure. System correctness is covered by `mars -selfcheck`: a headless run driving the real `App` with **no mocks** — real PTYs, a real daemon over a real socket, covering detach/reattach, PTY survival, client takeover, and the directive re-ask cap end-to-end; screen assertions parse a real `vt100` interpreter rather than substring-matching raw bytes. The *agent* evaluation below measures answer quality, and turns one knob: the memory variant.

Axis A — command translation. Given an English request and terminal context, is the produced command correct? We score with an LLM judge (Zheng et al., 2023) against a gold set drawn from NL2Bash-style data (Lin et al., 2018) and the logged real distribution. The distinctive measurement is the **corrective-memory lift**: replay a previously-corrected request and measure the wrong→right rate. Memory here is the user's own commands.

Axis B — self-knowledge and reconfigure. Can the agent answer questions about MARS and propose correct config edits ("make the accent orange" → the right knob)? We score with a Claude oracle against a gold set; memory here is MARS's own docs and registry.

Ablation and cost. Both axes run across the four memory variants (`none`/`history`/`docs`/`full`) and across model tiers. Cost is reported in **tokens per session-hour** (from the log's session boundaries), not dollars, because the free-tier multi-provider setting makes a dollar figure volatile.

Tables 2–3 give the reporting shape. Our hypotheses (not results): command memory lifts open-model accuracy toward the closed-model baseline; doc memory sharply raises self-Q&A while a memoryless model confabulates features; and memory's token overhead is modest against a session-hour's baseline traffic.

Axis / Task	Memory	Score%	Δ
A: translate (T2 open)	none	TBD	—
	history	TBD	TBD
A: translate (T3 closed)	none	TBD	—
	history	TBD	TBD
B: self-Q&A	none	TBD	—
	docs	TBD	TBD
B: reconfigure	none	TBD	—
	docs	TBD	TBD

Table 2: Accuracy by axis, model tier, and memory variant. [TODO: fill from eval]

Memory	Wrong→right	Right→wrong	Tok/sess-hr
none	—	—	TBD
history	TBD	TBD	TBD
docs	TBD	TBD	TBD
full	TBD	TBD	TBD

Table 3: Corrective-memory lift and token cost. The right→wrong column audits whether memory ever *harms* a correct answer. [TODO: fill from eval]

Limitations. (1) **No exit-code evaluation**: commands are typed into a PTY fire-and-forget, so Axis A is judged on command *text*, not observed success. (2) **Tokens, not dollars**, for the reason above. (3) **BM25, not embeddings**: recall is bounded by lexical overlap, a deliberate choice at sub-1k corpus size. (4) **No intent-classification metric**: the bar routes by sigil, so routing is deterministic by construction.

7 Comparison, Availability, Conclusion

Table 4 situates MARS. tmux+Claude Code has a daemon and an agent, but the agent runs in one pane and cannot see the adjacent buffer or accumulate the user's cross-session command memory. Warp does AI command search well but sees its own blocks, not your editor, and is not persistence-first. Cursor runs its intelligence on the laptop while remote state lives on the box, and dies with its window. Matching MARS's memory augmentation would require each to grow a persistent, terminal-resident memory their architectures do not host. MARS's posture toward strong code agents is to be the substrate they run *inside*, giving them sight of the other panes (Yao et al., 2023; Schick et al., 2023), not to reimplement them.

Availability. MARS is MIT-licensed and installs with `cargo install mars-terminal` (binary

Capability	MARS	tmux+CC	Warp	Cursor
Cross-pane sight	✓	—	partial	—
Persistent daemon	✓	✓	—	—
User-command memory	✓	—	partial	—
Watch while detached	✓	—	—	—
Key never leaves home	✓	—	—	—
Single local binary	✓	—	—	—

Table 4: MARS versus tmux+Claude Code (CC), Warp (Warp, 2024), and Cursor (Anysphere, 2024).

`mars`; source `https://github.com/anjishnu/mars`); it needs Rust $\geq$ 1.85 and runs on macOS and Linux, working out of the box with a free-tier key from any supported provider or any OpenAI-compatible endpoint. The demonstration shows, live: recalling a previously-corrected command from memory, triaging a failure across two panes, watching a training run while detached and reattaching to its verdict, and the key-never-leaves-home broker serving an agent on a keyless remote GPU box.

Conclusion. MARS argues that a developer's LLM assistant should live *inside* the terminal and that its defining capability is *memory*: retrieval over the user's own commands and over the tool's own ground truth, accumulated by an agent that sees every pane and persists across detach. Built with tasteful minimalism and kept honest by the keymap invariant, MARS turns a blind, amnesiac chat window into a terminal-resident colleague that remembers. Running the specified evaluation, and filling the numbers that read TBD, is the immediate next step.

References

Anysphere. 2024. Cursor: The AI code editor. `https://cursor.com`. Accessed July 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. 2017. Program synthesis from natural language using recurrent neural networks. *University of Washington Department of Computer Science and Engineering, Tech. Rep. UW-CSE-17-03-01*.

Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Warp. 2024. Warp: The intelligent terminal. `https://www.warp.dev`. Accessed July 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.